| | | | |
|---|---|---|---|
| *Akademia Górniczo-Hutnicza w Krakowie* | | | |
| Faculty: AGH | Academic Year 2025/2026 | Year of study | Fields of science: |
| Student(s) name: | Simon GANIER-LOMBARD & Daniil STEPANOV | | |
| Course: | **Real Time Operating Systems in Control Applications** | | |
| Exercise nr: | **RTOS-08L** | | |
| Exercise subject: | **FreeRTOS: communication with queues** | | |
| Lecturer: | dr inż. Grzegorz Hayduk | | |
| Date: 16.06.2026 | | | Grade: |

## 1 Exercise purpose

The goal of this exercise is to learn how two FreeRTOS tasks talk to each other with a queue.
We create three queues and pass three kinds of data through them: a single integer, an array of five integers, and a string. One task sends, the other task receives.
We also test what happens when the receive does not block and when the send and receive timeouts have different values, so the queue is sometimes empty and sometimes full.

## 2 Assumptions / Theory

A queue in FreeRTOS is a buffer that holds a fixed number of items of a fixed size. One task puts items in, another task takes them out, and the kernel handles the locking.
The main functions are:
- xQueueCreate takes the queue length (number of items) and the size of one item, and returns a QueueHandle_t.
- xQueueSend takes the queue handle, a pointer to the data to copy in, and a timeout in ticks. If the queue is full it waits up to the timeout, then returns a value other than pdPASS.
- xQueueReceive takes the queue handle, a pointer to a buffer for the data, and a timeout in ticks. If the queue is empty it waits up to the timeout, then returns a value other than pdPASS.
The timeout uses pdMS_TO_TICKS to turn milliseconds into ticks. A timeout of 0 means the call returns at once without waiting. The queue copies the data by value, so the sender and receiver do not share the same memory.

## 3 Description of implementation

We start from the FreeRTOS_lab project and add one sender task and one receiver task that share three queues. Each queue holds one item, so the sender and receiver have to take turns.
- xQueues[0] carries a single int.
- xQueues[1] carries an array of five int.
- xQueues[2] carries a string of up to fifty characters.
Task 1 builds the three values, waits SEND_DELAY, then sends each one with a timeout of SEND_TIMEOUT. If a queue is still full when the timeout ends, the send fails and the task prints a Send timeout message. Task 2 waits RECEIVE_DELAY between rounds and receives each queue with a

timeout of RECEIVE_TIMEOUT. If a queue is empty when the timeout ends, the receive fails and the task prints a queue empty message.
The full source of main.c is shown below.

*main.c (send task, receive task, and queue setup):*

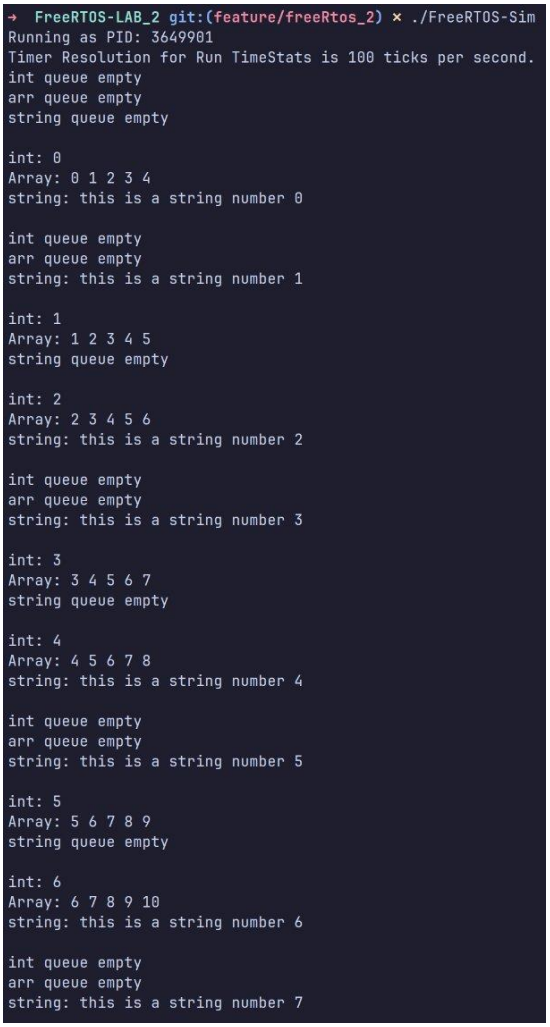
```
/* Standard includes. */
#include <stdio.h>

#include "FreeRTOS.h"
#include "queue.h"
#include "task.h"

/* Some definitions */
#define mainQUEUE_LENGTH                ( 1 )

// time based define
#define SEND_DELAY                ( 2000 ) // 2000
#define RECEIVE_DELAY                ( 500 )

#define SEND_TIMEOUT    pdMS_TO_TICKS( 1000 ) // 1000
#define RECEIVE_TIMEOUT pdMS_TO_TICKS( 500 )


#define mainARRAY_LENGTH                ( 5 )
#define mainSTRING_LENGTH                ( 50 )



/*-----------------------------------------------------------*/
static void xStartTask_1( void *pvParameters )
{
    QueueHandle_t *xQueues = ( QueueHandle_t * ) pvParameters;
    int xInteger = 0;
    int xArray[ mainARRAY_LENGTH ];
    char pcString[ mainSTRING_LENGTH ];
    int xIndex;
    const TickType_t xDelay = pdMS_TO_TICKS( SEND_DELAY );

    for( ;; ) {
        for( xIndex = 0; xIndex < mainARRAY_LENGTH; xIndex++ ) {
            xArray[ xIndex ] = xInteger + xIndex;
        }

        snprintf( pcString, mainSTRING_LENGTH,
                  "this is a string number %d", xInteger );

        vTaskDelay( xDelay );

        // maximum delay, wait infinite time until has space.
        // xQueueSend( xQueues[ 0 ], &xInteger, portMAX_DELAY );
        // xQueueSend( xQueues[ 1 ], xArray, portMAX_DELAY );
        // xQueueSend( xQueues[ 2 ], pcString, portMAX_DELAY );

        // now timeout a certain amount of time, if the queue is full, then skip sending.
        if( xQueueSend( xQueues[ 0 ], &xInteger, SEND_TIMEOUT ) != pdPASS ) {
            printf( "Send timeout INTEGER\r\n" );
        }
        if (xQueueSend( xQueues[ 1 ], xArray, SEND_TIMEOUT ) != pdPASS ) {
            printf( "Send timeout ARRAY\r\n" );
        }
        if (xQueueSend( xQueues[ 2 ], pcString, SEND_TIMEOUT ) != pdPASS ) {
            printf( "Send timeout STRING\r\n" );
        }

        xInteger++;
    }
}


/*-----------------------------------------------------------*/
static void xStartTask_2( void *pvParameters )
{
    QueueHandle_t *xQueues = ( QueueHandle_t * ) pvParameters;
    int xInteger;
    int xArray[ mainARRAY_LENGTH ];
    char pcString[ mainSTRING_LENGTH ];
    int xIndex;
    const TickType_t xDelay = pdMS_TO_TICKS( RECEIVE_DELAY );
```

```
    for( ;; ) {

        // 0 timeout mean return imediate, RECEIVE_TIMEOUT mean wait a certain amount of time, if the queue is
empty +time is passed, then skip receiving.
        //if( xQueueReceive( xQueues[ 1 ], xArray, 0 ) == pdPASS )
        if( xQueueReceive( xQueues[ 0 ], &xInteger, RECEIVE_TIMEOUT) == pdPASS ) {
            printf( "int: %d\r\n", xInteger );
        } else {
            printf( "int queue empty\r\n" );
        }

        //if( xQueueReceive( xQueues[ 1 ], xArray, 0 ) == pdPASS )
        if( xQueueReceive( xQueues[ 1 ], xArray, RECEIVE_TIMEOUT) == pdPASS ) {
            printf( "Array:" );
            for( xIndex = 0; xIndex < mainARRAY_LENGTH; xIndex++ ) {
                printf( " %d", xArray[ xIndex ] );
            }
            printf( "\r\n" );
        } else {
            printf( "arr queue empty\r\n" );
        }

        // if( xQueueReceive( xQueues[ 2 ], pcString, 0 ) == pdPASS
        if( xQueueReceive( xQueues[ 2 ], pcString, RECEIVE_TIMEOUT ) == pdPASS ){
            printf( "string: %s\r\n", pcString );
        } else {
            printf( "string queue empty\r\n" );
        }

        printf( "\r\n" );
        fflush( stdout );
        vTaskDelay( xDelay );
    }
}

/*-----------------------------------------------------------*/

int main ( void )
{
    QueueHandle_t xQueues[ 3 ];

    xQueues[ 0 ] = xQueueCreate( mainQUEUE_LENGTH, sizeof( int ) );
    xQueues[ 1 ] = xQueueCreate( mainQUEUE_LENGTH,
                                 sizeof( int ) * mainARRAY_LENGTH );
    xQueues[ 2 ] = xQueueCreate( mainQUEUE_LENGTH,
                                 mainSTRING_LENGTH * sizeof( char ) );

    /* Create the tasks defined within this file. */
    xTaskCreate( xStartTask_1, "Task_1", configMINIMAL_STACK_SIZE,
                 xQueues, 1, NULL );
    xTaskCreate( xStartTask_2, "Task_2", configMINIMAL_STACK_SIZE,
                 xQueues, 1, NULL );

    /* Start the scheduler itself. */
    vTaskStartScheduler();

    /* Should never get here unless there was not enough heap space to create
    the idle and other system tasks. */
    return 0;
}
/*-----------------------------------------------------------*/
```

**We ran the program with two parameter sets to test the two timeout cases asked for in the lab.**
First set: the sender is slow (SEND_DELAY 2000 ms) and the receiver is fast (RECEIVE_DELAY 500 ms). The receiver checks the queues more often than the sender fills them, so the queues are often empty and the receiver hits its RECEIVE_TIMEOUT. The output shows queue empty messages.

*Parameters for the first run (receive timeout, queues often empty):*

```
#define mainQUEUE_LENGTH    ( 1 )
#define SEND_DELAY          ( 2000 )
#define RECEIVE_DELAY       ( 500 )
#define SEND_TIMEOUT        pdMS_TO_TICKS( 1000 )
#define RECEIVE_TIMEOUT     pdMS_TO_TICKS( 500 )
#define mainARRAY_LENGTH    ( 5 )
#define mainSTRING_LENGTH   ( 50 )
```

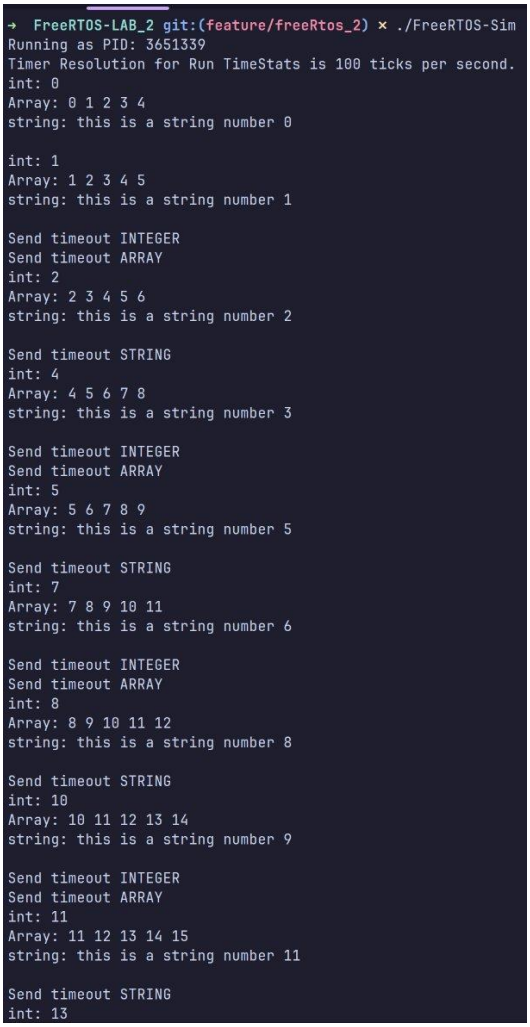



*First run: the receiver finds the queues empty between the slow sends and prints queue empty.*

Second set: the sender is fast (SEND_DELAY 200 ms) with a short SEND_TIMEOUT of 100 ms, while the receiver still waits 500 ms between rounds. The sender fills the one-item queues faster than the receiver empties them, so the queues are often full and the sender hits its SEND_TIMEOUT. The output shows Send timeout messages for INTEGER, ARRAY, and STRING.

*Parameters for the second run (send timeout, queues often full):*

```
#define mainQUEUE_LENGTH    ( 1 )
#define SEND_DELAY          ( 200 )
#define RECEIVE_DELAY       ( 500 )
#define SEND_TIMEOUT        pdMS_TO_TICKS( 100 )
#define RECEIVE_TIMEOUT     pdMS_TO_TICKS( 500 )
#define mainARRAY_LENGTH    ( 5 )
#define mainSTRING_LENGTH   ( 50 )
```




*Second run: the sender cannot place new items in the full queues and prints Send timeout.*

## 4 Conclusions

This exercise showed how a FreeRTOS queue moves data between two tasks. The queue copies each item by value, so the sender and receiver never share memory, and it works the same way for an integer, an array, or a string. The only difference is the item size given to xQueueCreate.

The timeout is the key point. With a one-item queue, the result depends on which task is faster. In the first run the sender was slow and the receiver was fast, so the queues were empty most of the time and the receive calls timed out. In the second run the sender was fast with a short send timeout, so the one-item queues filled up and the send calls timed out instead. A receive timeout of 0 returns at once without waiting, which is the non-blocking case from the lab.

So the same code shows two opposite behaviours just by changing the delays and timeouts. The queue length and the relation between send rate and receive rate decide whether a task waits, skips, or blocks.